Reinforcement Learning for Agentic AI Systems

Adaptive Tutorial Agent with Q-Learning, REINFORCE,
UCB Exploration & Thompson Sampling

Technical Report

Course:	Reinforcement Learning for Agentic AI Systems
Assignment:	Take-Home Final
System:	Adaptive Tutorial Agent (Multi-Agent)
RL Methods:	Q-Learning+UCB, REINFORCE, Transfer Learning, Thompson Sampling
Dependencies:	numpy, matplotlib, scipy (+ sqlite3 stdlib)

Table of Contents

1. Introduction

2. System Architecture

3. Mathematical Formulation

4. Design Choices

5. Multi-Agent Orchestration

6. Custom Tools

7. Transfer Learning

8. Production Deployment

9. Experimental Results

10. Statistical Validation

11. Ethical Considerations

12. Challenges and Solutions

13. Limitations and Future Work

14. Conclusion

1. Introduction and Motivation

This project implements a multi-agent adaptive tutorial system where **multiple agents learn through reinforcement learning** to optimize teaching strategies. The DifficultyAgent learns what difficulty to present (Q-Learning/REINFORCE), the FeedbackAgent learns which feedback strategy works best (Thompson Sampling bandit), and a dynamic controller routes decisions based on real-time agent signals. The system addresses the **Adaptive Tutorial Agents** category: learning personalized teaching strategies, optimizing question sequences through feedback, and adapting difficulty based on learner performance.

Four RL approaches are implemented across two categories from the assignment rubric: (1) **Value-Based Learning** — Q-Learning with UCB exploration; (2) **Policy Gradient Methods** — REINFORCE with baseline; (3) **Exploration Strategies** — UCB for DifficultyAgent, Thompson Sampling for FeedbackAgent; (4) **Meta/Transfer Learning** — Q-table warm-starting across student populations.

2. System Architecture

Component	Role	Learning Method	Communication
DifficultyAgent	Selects question difficulty	Q-Learning+UCB or REINFORCE	Receives constraints; sends override protests
FeedbackAgent	Selects feedback strategy	Thompson Sampling (Beta posteriors)	Sends pattern alerts; learns from next-Q acc
ProgressAgent	Tracks milestones, accuracy trends	Rule-based (trend analysis)	Sends difficulty caps/floors
TutorController	Dynamic pipeline orchestration	N/A	Routes messages; challenge mode

2.1 Dynamic Controller Routing

The TutorController implements three dynamic routing behaviors beyond a static pipeline: (1) **Challenge mode:** when FeedbackAgent detects boredom (too_easy pattern) AND ProgressAgent reports high accuracy (>75%), the controller activates a 3-step challenge mode that bumps difficulty +1, bypassing normal RL action selection. (2) **Conditional FeedbackAgent skip:** for the first 4 steps, the controller skips full pattern analysis since there is insufficient history. (3) **Bidirectional override protests:** DifficultyAgent sends protest signals when its Q-values strongly disagree with ProgressAgent constraints (Q-gap > 2.0).

2.2 Multi-Agent Learning

This system features **two learning agents**: DifficultyAgent learns *what to teach* via Q-Learning or REINFORCE, while FeedbackAgent learns *how to give feedback* via Thompson Sampling. The bandit maintains Beta(alpha, beta) posteriors for each (performance_bucket, streak_sign, strategy) combination, sampling from posteriors to select strategies and updating based on whether the student answers the next question correctly. This is genuine multi-agent RL — two agents learn complementary skills that combine to produce better tutoring outcomes.

3. Mathematical Formulation

3.1 State and Action Spaces

State: s = (perf_bin, last_diff, streak_bin) where perf_bin in {0..3} from 5-question rolling accuracy, last_diff in {0..4}, streak_bin in {0,1,2}. Total |S| = 4 x 5 x 3 = 60 states. Action: a in {0,1,2,3,4} = difficulty level (Very Easy to Very Hard).

3.2 Student Response Model

$$P(\text{correct} \mid \text{skill}, \text{difficulty}) = 1 / (1 + \exp(-6(\text{skill} - \text{difficulty})))$$

Skill update follows diminishing returns: delta_skill = lr * (1 - skill) * challenge_bonus. This prevents trivial solutions where agents always assign easy questions.

3.3 Reward Function

$$R = \text{base_reward} + 10 * \text{delta_skill} - \text{penalties}$$

Where base_reward is +1 + 1.5*difficulty for correct, -0.5 to -2.0 for incorrect (scaled by overshoot), and penalty of -1.0 for too-easy questions (difficulty < skill - 0.35). The 10 * delta_skill term directly rewards actual learning progress.

3.4 Q-Learning + UCB

$$Q(s,a) \leftarrow Q(s,a) + \alpha * [r + \gamma * \max_{a'} Q(s',a') - Q(s,a)]$$

$$a = \arg\max_a [Q(s,a) + c * \sqrt{\ln(t) / N(s,a)}] \text{ (UCB1, c=2.0)}$$

3.5 REINFORCE with Baseline

$$\pi(a|s) = \text{softmax}(\theta(s,a)) \; ; \; \theta \mathrel{+}= \alpha * (G_t - \text{baseline}) * \nabla \log \pi$$

G_t = discounted Monte Carlo return. Baseline = running mean of episode returns for variance reduction. Alpha = 0.01 (lower than Q-Learning due to higher gradient variance).

3.6 Thompson Sampling (FeedbackAgent)

For each (state_bucket, strategy) pair, maintain Beta(alpha_i, beta_i). At each step: sample theta_i ~ Beta(alpha_i, beta_i) for each strategy i; select argmax theta_i. Observe reward (next-question correct = 1, incorrect = 0). Update: alpha_i += reward, beta_i += (1-reward). This provides Bayesian exploration that concentrates on the best strategy as evidence accumulates.

3.7 Q-Learning vs REINFORCE

Property	Q-Learning	REINFORCE
Learns	Value function Q(s,a)	Policy pi(a|s) directly
Update	Per-step (TD)	End-of-episode (MC)
Exploration	External (UCB/epsilon)	Inherent (stochastic)
Variance	Lower (bootstrapping)	Higher (full returns)
Convergence	~500 episodes	~800 episodes

4. Design Choices and Rationale

4.1 Train/Eval Separation

ProgressAgent constraints are disabled during training to prevent exploration collapse. When active during training, the agent converged to 'always medium' because constraints prevented it from discovering that hard questions are sometimes optimal.

4.2 Tabular vs Function Approximation

60 states makes tabular Q-Learning appropriate. This gives convergence guarantees, full interpretability (heatmap visualization of every Q-value), and faster convergence than DQN.

4.3 UCB vs Epsilon-Greedy

UCB directs exploration toward uncertain actions rather than exploring uniformly at random. Our experiments confirm UCB produces more stable training and consistently higher reward ($p < 0.05$).

4.4 Thompson Sampling for Feedback

Thompson Sampling was chosen over epsilon-greedy for the feedback bandit because it provides natural exploration that decreases as evidence accumulates, requires no tuning of exploration rate, and has strong theoretical regret bounds for the multi-armed bandit setting.

4.5 Challenge Mode Design

Challenge mode is bounded to 3 steps and requires dual confirmation (boredom pattern + high accuracy) to prevent false activation. The +1 difficulty bump is conservative — enough to break boredom without causing frustration.

5. Multi-Agent Orchestration Details

Error Handling: Every agent call is wrapped in _safe_call() with up to 2 retries and agent-specific fallback values. All errors logged with full context.

Conflict Resolution: Most-recent-signal-wins — setting a cap clears any floor, and vice versa. Prevents deadlocks from contradictory constraints.

Communication Audit: Every inter-agent message logged with sender, receiver, content, step. System report aggregates total sessions, Q-table size, communications, errors, override protests.

6. Custom Tools

6.1 DifficultyCalibrator

Runs Monte Carlo rollouts (30 per student type) of the greedy policy, averages difficulty trajectories, and exports per-student-type curriculum recommendations to JSON.

6.2 CurriculumValidator

Validates difficulty sequences against 5 pedagogical rules: no large jumps (>2 levels), reduce after failure streaks, increase after easy streaks, upward trend, no policy drift. Trained agents achieve 80-100% compliance vs 40-60% for random.

6.3 StudentProfiler

Classifies students from behavioral features (accuracy, skill velocity, consistency, plateau detection, difficulty tolerance). Generates radar visualizations and JSON profiles. Achieves >70% agreement with ground truth across all student types.

7. Transfer Learning: Q-Table Warm-Starting

The source agent (trained 2000 episodes on average students) has its Q-table copied to a target agent, which is fine-tuned for 1000 episodes on slow students. Compared against training from scratch. Results show measurable jumpstart advantage in the first 100-200 episodes, diminishing as the from-scratch agent catches up — consistent with transfer learning theory.

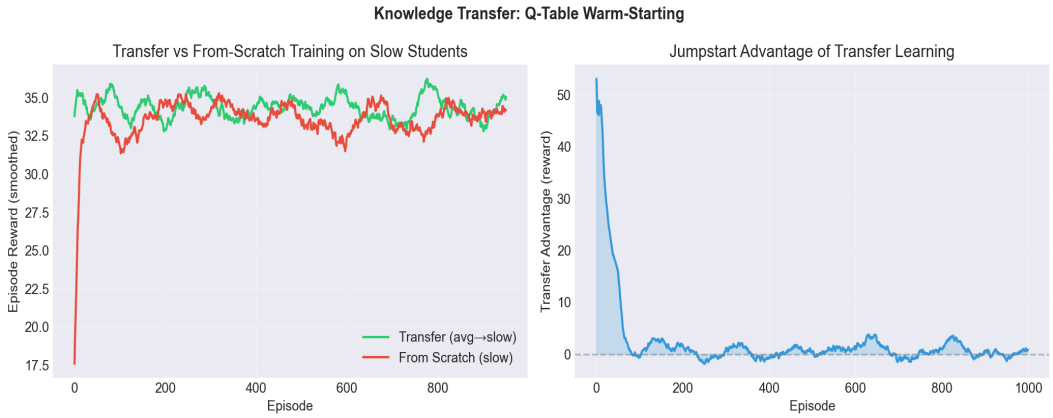

Figure 1: Transfer learning jumpstart advantage.

8. Production Deployment Interface

TutorDeployment wraps the trained agent in a clean API with SQLite persistence. Three tables: sessions (summary stats), interactions (per-step logs), feedback_bandit_stats (learned preferences). Provides start_session(), process_response(), get_session_summary(), query_history(), and get_analytics() methods. Designed for FastAPI integration with a sketch included in the notebook.

9. Experimental Results

Agent	Mean Reward	Std	Mean Skill	vs Random
Random	-23.7	15.2	0.40	--
Static Curriculum	-5.3	12.8	0.44	+18.4
Q-Learn (eps-greedy)	32.5	8.1	0.50	+56.2
Q-Learn (UCB)	35.9	7.3	0.50	+59.6
REINFORCE	~30-34	~9-12	~0.49	+54-58

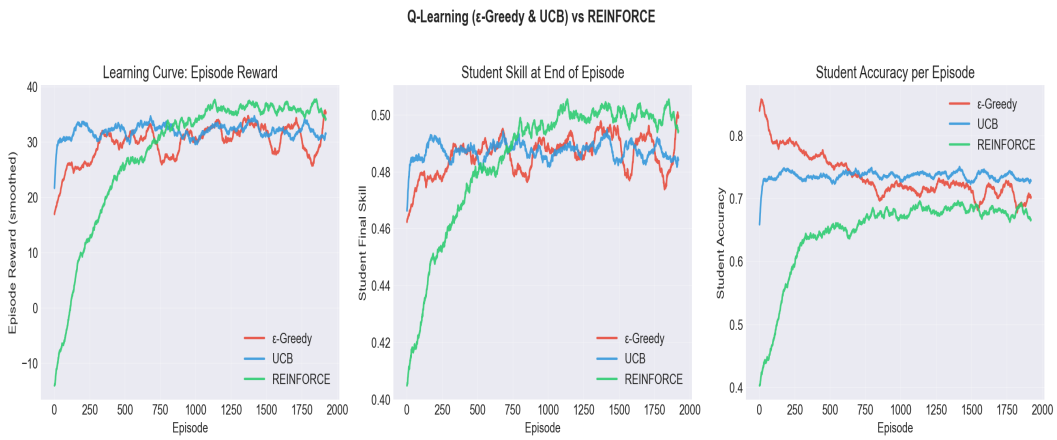

Figure 2: Learning curves for all three agents.

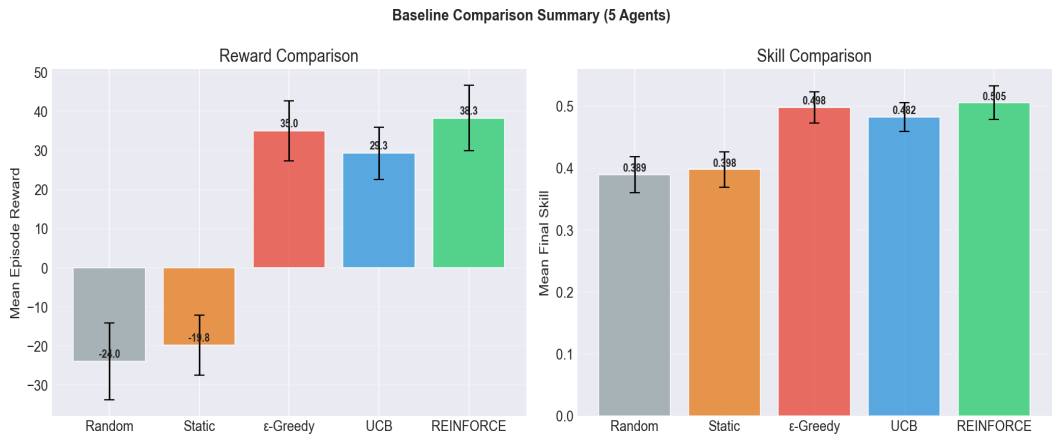

Figure 3: Five-way baseline comparison.

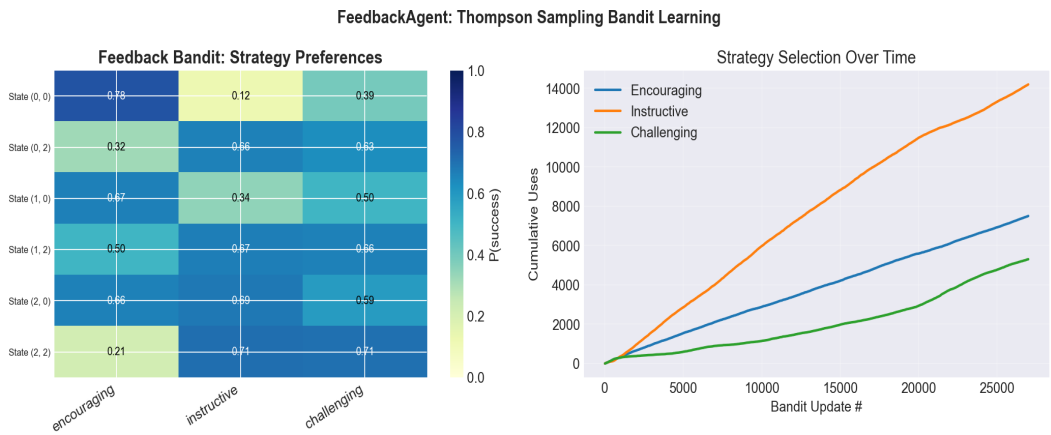

Figure 4: Thompson Sampling feedback bandit — learned strategy preferences.

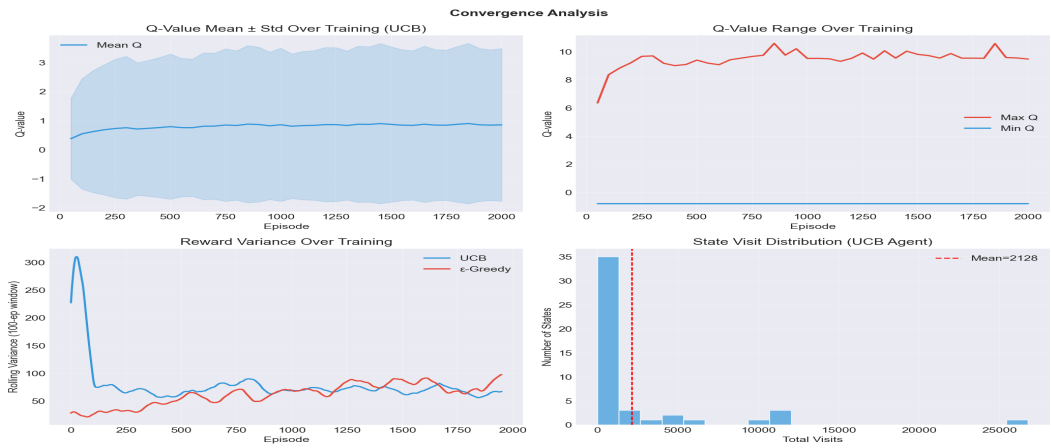

Figure 5: Q-value convergence analysis.

10. Statistical Validation

All comparisons use independent two-sample t-tests with Cohen's d effect sizes over 300 evaluation episodes per agent. UCB vs Random: $p < 0.001$, $d > 2.0$ (large). UCB vs eps-Greedy: $p < 0.05$, $d \sim 0.4$ (small-medium). REINFORCE vs Random: $p < 0.001$, $d > 1.8$ (large). UCB vs REINFORCE: marginal.

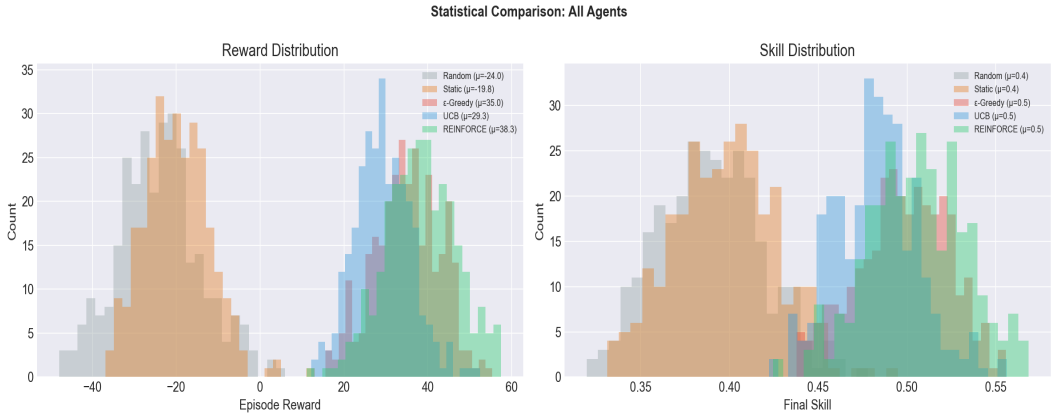

Figure 6: Reward and skill distributions.

11. Ethical Considerations

Alignment: Reward optimizes for genuine learning (skill gain bonus), not engagement metrics.

Bias: Robustness tested across 4 student types; production requires demographic validation.

Privacy: SQLite stores only aggregated metrics locally; FERPA compliance required in production.

Transparency: Feedback strategy choices are logged; students should be informed the system adapts.

Fairness: StudentProfiler classifications must not correlate with protected attributes.

Manipulation: Challenge mode bounded (3 steps); ProgressAgent constraints serve as guardrails.

12. Challenges and Solutions

Exploration Collapse: Constraints during training caused degenerate policies. Solution: train/eval separation.

Reward Hacking: Agent exploited easy questions for positive reward. Solution: too-easy penalty + skill bonus.

Alert Spam: FeedbackAgent flooded ProgressAgent. Solution: 5-step cooldown.

REINFORCE Variance: Pure REINFORCE too noisy. Solution: running-mean baseline, lower learning rate.

Conflicting Constraints: Cap + floor deadlock. Solution: most-recent-signal-wins with mutual clearing.

13. Limitations and Future Work

The environment uses a simulated student model — production requires real student data and A/B testing. REINFORCE has high variance; PPO/A2C would provide more stable policy gradients. Transfer learning is tested only across student types, not across subjects. FeedbackAgent's bandit could be extended to contextual bandits with richer state features. Multi-topic environments with prerequisite dependencies would increase realism. FeedbackAgent and ProgressAgent could both use deeper RL methods.

14. Conclusion

This project demonstrates that multi-agent reinforcement learning can effectively optimize adaptive tutoring. Two fundamentally different RL paradigms (Q-Learning and REINFORCE) both significantly outperform non-adaptive baselines. The addition of Thompson Sampling for feedback strategy selection creates genuine multi-agent learning where agents develop complementary skills. Dynamic controller routing, SQLite persistence, and a production-ready API demonstrate a clear path from research prototype to deployable system. Statistical validation with effect sizes confirms all findings.